# OPLOGS-01: OpenPipeline Fundamentals

> **Series:** OPLOGS — OpenPipeline Logs | **Notebook:** 1 of 8 | **Created:** December 2025 | **Last Updated:** 10/14/2026

## Understanding the Unified Data Ingestion Framework

This notebook introduces OpenPipeline, Dynatrace's unified data processing framework for logs, traces, metrics, and events.

> 🆕 **New Addition (March 2026)**
>
> **Companion Series**
> - **OPMIG** — Migrating from classic logs to OpenPipeline? Start with **OPMIG-01: Why Migrate**
> - **OPIPE** — Ready to process spans, metrics, and events? Continue to **OPIPE-01: OpenPipeline as a Multi-Scope Platform**
> - **SPANS** — Need to query and analyze distributed traces? See **SPANS-01: Fundamentals**

---

## Table of Contents


1. What is OpenPipeline?
2. OpenPipeline Architecture
3. Exploring Your OpenPipeline Data
4. Key OpenPipeline Fields
5. Data Sources Explained
6. Pipeline Stages
7. Environment Summary

8. 📝 Summary
9. ➡️ Next Steps
10. 📚 References

---

## Prerequisites

- ✅ Access to a Dynatrace environment with log data
- ✅ DQL query permissions (viewer role minimum)
- ✅ Basic understanding of log management concepts

## 1. What is OpenPipeline?

**OpenPipeline** is Dynatrace's unified data ingestion and processing framework that replaces classic log ingestion. It provides:

- **Unified Processing**: Single framework for logs, metrics, traces, and business events
- **Real-time Transformation**: Parse, enrich, mask, and route data at ingestion
- **Grail Storage**: Direct integration with Dynatrace's data lakehouse
- **Flexible Routing**: Send data to different buckets with custom retention
- **Cost Control**: Drop unnecessary data before storage

### OpenPipeline vs the Classic Log Pipeline

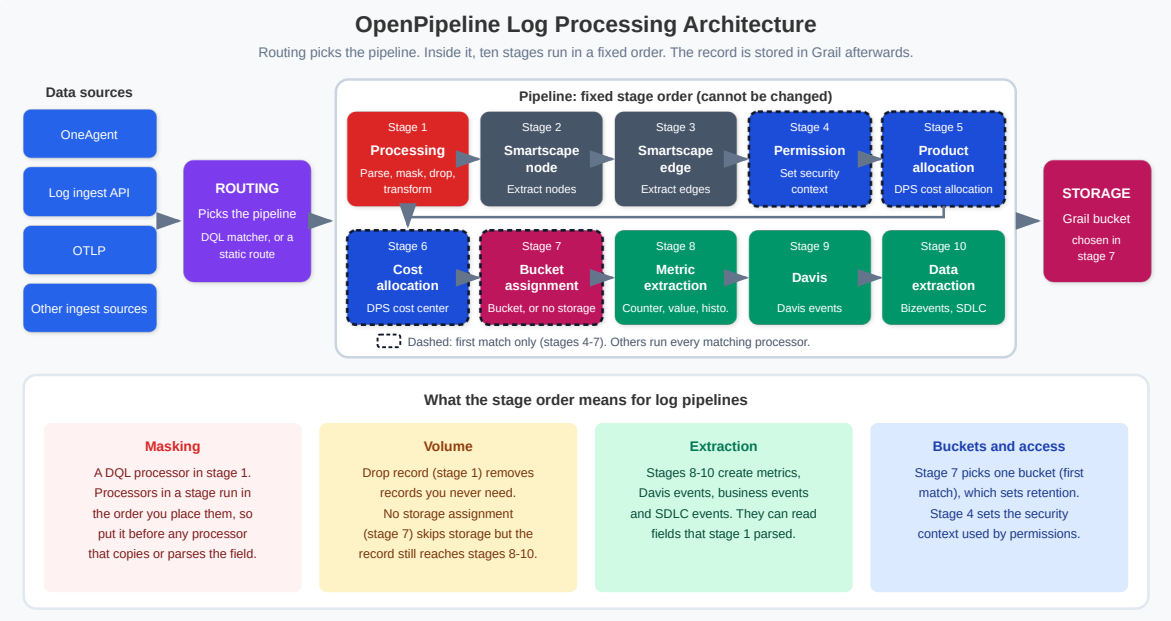
On Grail, OpenPipeline replaces the **classic log pipeline** — log processing rules under *Settings > Log Monitoring*. Both process records **before** they are stored; the differences are scope, language and structure:

| Feature | Classic log pipeline | OpenPipeline |
|---|---|---|
| Data Processing | Processing rules at ingest, before storage | Pipelines at ingest, before storage |
| Signal types | Logs and business events | Logs, spans, metrics, events, business events, and more |
| Rule language | Processing-rule commands with DPL | DQL processors with DPL, plus no-code processors |

| Feature | Classic log pipeline | OpenPipeline |
|---|---|---|
| Routing | One ordered list of rules | Dynamic routing to many pipelines, pipeline groups |
| Data Masking | Mask / drop attributes in a rule | DQL processor with DPL ( `replacePattern` ), `replaceString` , `ipMask` |
| Extraction | Log metrics | Metric, Davis event, business event, SDLC event, Smartscape stages |
| Retention | Bucket per record | Bucket assignment stage, or No storage assignment |

> **Sources:** Log processing with classic pipeline (DT docs) — *"Log processing occurs as log data arrives in the Dynatrace SaaS environment and before it is written to disk (stored)."*

## 2. OpenPipeline Architecture




> **Stage model (verified 09/24/2026):** data flows Ingest → Routing → pipeline → Storage. Inside a pipeline, per *Processing in OpenPipeline*, "The sequence of stages is fixed for all pipelines and cannot be modified." Masking, dropping, parsing and transformation are processors in the first stage, **Processing**; **Bucket assignment** comes seventh, and

> **Metric extraction**, **Davis** and **Data extraction** run after it. The diagram shows all ten stages in that order; stages 4–7 run only the first matching processor.
>
> **Sources:** Processing in OpenPipeline (DT docs)

## 3. Exploring Your OpenPipeline Data

Let's start by discovering what data sources and pipelines are active in your environment.

```
// Discover data sources feeding OpenPipeline
fetch logs, from: now() - 1h
| summarize {log_count = count()}, by: {dt.openpipeline.source}
| sort log_count desc
```

```
// See which pipelines are processing your logs
fetch logs, from: now() - 1h
| summarize {log_count = count()}, by: {dt.openpipeline.pipelines}
| sort log_count desc
```

```
// Check storage bucket distribution
fetch logs, from: now() - 1h
| summarize {log_count = count()}, by: {dt.system.bucket}
| sort log_count desc
```

## 4. Key OpenPipeline Fields

OpenPipeline adds metadata fields to every log record:

### Pipeline Metadata

| Field | Description | Example |
|---|---|---|
| `dt.openpipeline.source` | How the log was ingested | `oneagent`, `/api/v2/logs/ingest`, `/api/v2/otlp/v1/logs` |

| Field | Description | Example |
| --- | --- | --- |
| `dt.openpipeline.pipelines` | Pipeline(s) that processed the log | `["logs:pipeline_Default_Pipeline_2798"]` |
| `dt.system.bucket` | Storage bucket name | `default_logs`, `custom_logs` |

## Core Log Fields

| Field | Description |
| --- | --- |
| `timestamp` | When the log was generated |
| `content` | The log message body |
| `loglevel` | Log severity (ERROR, WARN, INFO, DEBUG, NONE) |
| `status` | Severity group derived from `loglevel`: ERROR (SEVERE, ERROR, CRITICAL, ALERT, FATAL, EMERGENCY), WARN, INFO (INFO, TRACE, DEBUG, NOTICE), NONE — use it for error-class counts |
| `log.source` | Source identifier (e.g., "Container Output") |
| `log.iostream` | Stream type (stdout, stderr) |

```
// View a sample log record with all key fields
fetch logs, from: now() - 1h
| fields timestamp, content, loglevel, status, log.source, log.iostream,
         dt.openpipeline.source, dt.openpipeline.pipelines, dt.system.bucket
| limit 5
```

```
// Analyze log levels in your environment
fetch logs, from: now() - 1h
| summarize {count = count()}, by: {loglevel}
| sort count desc
```

# 5. Data Sources Explained

## OneAgent ( `oneagent` )

Logs collected automatically by Dynatrace OneAgent from:
- Container stdout/stderr
- Process log files
- System logs

## Log Ingest API ( `/api/v2/logs/ingest` )

Logs sent directly via the Dynatrace API:
- Custom application logs
- Third-party integrations
- Cloud provider logs (AWS, Azure, GCP)

### Delivery reliability

The Log Ingest API acknowledges accepted data with **HTTP 204 No Content** (empty body). When a request fails on a *retryable* response code (each endpoint documents which codes are retryable), the **client is responsible for retrying with exponential backoff** — Dynatrace does not retry on the sender's behalf. The ActiveGate that serves the endpoint buffers ingested data to an on-disk queue (default **300 MB**, configured by `disk_queue_max_size_mb`; location by `disk_queue_path`) and forwards it to Dynatrace in batches; when that queue fills, the API returns `503 Usable space limit reached` as a back-pressure signal — increase `disk_queue_max_size_mb` if this recurs under normal load. Cloud-forwarder integrations (e.g. Amazon Data Firehose) implement their own buffering and smart-retry against the OpenPipeline endpoint.

## OTLP ( `/api/v2/otlp/v1/logs` )

OpenTelemetry Protocol logs:
- OpenTelemetry Collector
- Fluent Bit with OTLP output
- Custom OTLP exporters

```
// Compare volume by data source
fetch logs, from: now() - 24h
| summarize {
    log_count = count(),
    unique_hosts = countDistinct(dt.entity.host)
  }, by: {dt.openpipeline.source}
| sort log_count desc
```

```
// Logs per hour by source (trend analysis)
fetch logs, from: now() - 24h
| makeTimeseries {log_count = count()}, by: {dt.openpipeline.source}, interval:
1h
```

## 6. Pipeline Stages Overview

A record first passes **Routing**, which picks the pipeline (a dynamic DQL matcher, or a static route for custom ingest sources). Inside the pipeline it then runs through a **fixed** sequence of ten stages — the order cannot be changed:

| # | Stage | What it does in a log pipeline | Processors | Executed |
|---|---|---|---|---|
| 1 | **Processing** | Parse values into fields, transform the schema, filter records, mask sensitive data | DQL, Add fields, Remove fields, Rename fields, Drop record, GeoIP lookup (Early Access), Inline lookup | All matches |
| 2 | Smartscape node | Extract Smartscape nodes from matching records | Smartscape node | All matches |
| 3 | Smartscape edge | Extract Smartscape edges from matching records | Smartscape edge | All matches |
| 4 | Permission | Apply a security context to matching records | Set security context | First match only |
| 5 | Product allocation | Assign product or application usage | DPS Cost Allocation - Product | First match only |
| 6 | Cost allocation | Assign cost-center usage | DPS Cost Allocation - Cost Center | First match only |

| # | Stage | What it does in a log pipeline | Processors | Executed |
|---|---|---|---|---|
| 7 | **Bucket assignment** | Choose the Grail bucket — or not to store the record | Bucket assignment, No storage assignment | First match only |
| 8 | Metric extraction | Extract metrics from matching records | Counter metric, Histogram metric, Value metric | All matches |
| 9 | Davis | Extract a Davis event and re-ingest it into another pipeline | Davis event | All matches |
| 10 | Data extraction | Extract a business event or SDLC event and re-ingest it | Business event, Software development lifecycle event | All matches |

Masking, dropping and parsing are not separate stages: they are processors in stage 1, and within a stage they run in the order you place them. Storage in the chosen Grail bucket happens after the pipeline, so a record's retention is set by stage 7 — which is also why the extraction stages (8–10) still see records assigned to **No storage assignment**.

> **Sources:** Processing in OpenPipeline (DT docs) — *"each processor output becomes the input for the next one"*, OpenPipeline processing examples (DT docs) — *"The record continues through all pipeline stages, including metric extraction, and is not stored only at the end."*

## 7. Environment Summary

Let's get a complete picture of your OpenPipeline environment:

```
// Complete environment summary
fetch logs, from: now() - 1h
| summarize {
    total_logs = count(),
    unique_sources = countDistinct(dt.openpipeline.source),
    unique_buckets = countDistinct(dt.system.bucket),
    unique_hosts = countDistinct(dt.entity.host),
    error_count = countIf(loglevel == "ERROR" OR status == "ERROR"),
    warn_count = countIf(loglevel == "WARN" OR status == "WARN")
  }
```

```
// Top log sources by entity
fetch logs, from: now() - 1h
| filter isNotNull(dt.entity.host)
| summarize {log_count = count()}, by: {host.name, log.source}
| sort log_count desc
| limit 15
```

---

## 📝 Summary

In this notebook, you learned:

✅ **What OpenPipeline is** - Dynatrace's unified data processing framework
✅ **Architecture** - Data flow from sources through processing to Grail
✅ **Key fields** - `dt.openpipeline.source`, `dt.openpipeline.pipelines`, `dt.system.bucket`
✅ **Data sources** - OneAgent, Log Ingest API, OTLP
✅ **Pipeline stages** - a fixed sequence: Processing (mask, drop, parse, transform) → … → Bucket assignment → Metric / Davis / Data extraction

---

## ➡️ Next Steps

Continue to **OPLOGS-02: Migration Guide** to learn how to migrate from classic log ingestion to OpenPipeline v2.0.

---

## 📚 References

- OpenPipeline (DT docs)
- Processing in OpenPipeline (DT docs)
- Grail (DT docs)
- Dynatrace Query Language (DT docs)
- Automatic log enrichment (DT docs) — *"a status attribute is created with a value that is a sum of loglevel values based on the following grouping"*
- Log ingestion API (DT docs) — delivery reliability: HTTP 204 success, retryable codes + exponential backoff, ActiveGate disk-queue buffering, `503 Usable space limit reached`

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*